

Hyper-ARM: Hypernetwork-Based Associative Recall Memory for Sample-Efficient Reinforcement Learning

Pierce Prange 1

Independent Researcher

Abstract

Deep Reinforcement Learning (RL) agents often struggle with perceptual aliasing and suffer from poor sample efficiency due to reliance on purely parametric memory. Inspired by challenges in automated AI playtesting agents for procedural content generation (PCG), we propose **Hyper-ARM** (Hypernetwork-based Associative Recall Memory). Hyper-ARM combines non-parametric episodic memory with hypernetwork-driven routing to create a Sparse Mixture of Experts in latent space. Crucially, we introduce a **Parallel Query Encoder** architecture that decouples perception from attention learning. Unlike standard Transformers or LSTMs, our architecture explicitly retrieves past successful action-value associations via a learned attention mechanism over a memory bank, utilizing **EQV (Encoder-Query-Value)** mechanics rather than standard QKV attention. This decoupling allows the agent to learn functional similarity independent of visual similarity, solving perceptual aliasing where standard models fail. We demonstrate validated accuracy gains on vision benchmarks (6% on Fashion-MNIST) with minimal parameter overhead. Furthermore, we outline a framework for RL evaluation on Atari 2600, showing preliminary evidence that stacking these blocks enables multi-hop retrieval without catastrophic forgetting.

1. Introduction

The success of Deep Q-Networks (Mnih et al., 2015) and Transformers (Vaswani et al., 2017) in RL has been predicated on the assumption that large parametric models can implicitly learn world dynamics and memory. However, recent work suggests that standard architectures struggle when visual observations are ambiguous (e.g., two identical corridors requiring different turns) or when long-term credit assignment is required (Schmidhuber, 2015).

This paper originates from an engineering constraint: developing an AI for *Color Switch*, a game where visually identical obstacles require different actions based on context, to facilitate in automated AI

playtesting for procedural content generation (PCG). Standard LSTMs and PPO agents failed to converge where a novel architecture succeeded. We identified that the bottleneck was not representation capacity, but **associative recall**. The agent needed to retrieve specific past experiences associated with a visual latent, rather than generalize via gradient descent alone.

We introduce **Hyper-ARM**, an architecture that treats memory retrieval as a spatial hashing problem in latent space. Key contributions include:

1. **Decoupled Perception & Attention:** A parallel query encoder structure that learns attention projections without disturbing the frozen perceptual keys, preventing memory invalidation.
2. **Temporal Context Integration:** Explicit handling of temporal dynamics in RL via frame stacking and optional recurrent branches, distinct from the static key generation.
3. **EQV Mechanism:** A reformulation of attention where the Key is derived from a static Encoder, allowing for stable spatial hashing in latent space.
4. **Visual vs. Functional Similarity:** We demonstrate that Hyper-ARM can retrieve different memories for visually identical states (disambiguation) and similar memories for visually distinct states (generalization).
5. **Parameter Efficiency:** We show that a lightweight Query Encoder can yield significant performance gains by leveraging non-parametric memory rather than increasing model depth.

2. Related Work

Episodic Control: Neural Episodic Control (NEC) (Pritzel et al., 2017) introduced differentiable memory tables. Hyper-ARM extends this by decoupling perception from action mapping via hypernetworks, preventing memory invalidation during updates.

Mixture of Experts: Shazeer et al. (2017) popularized Sparse MoE in NLP. Our work applies this to RL policy heads, where routing is determined by binding affinity to latent centroids rather than learned gating networks.

World Models: Ha & Schmidhuber (2018) proposed learning latent dynamics. Hyper-ARM complements this by grounding predictions in retrieved memories rather than hallucinated latents.

Representation Stability: Recent work in self-supervised learning (e.g., BYOL) focuses on preventing representational collapse. Hyper-ARM addresses a complementary stability issue: preventing *memory address* collapse by freezing the key-generating encoder.

3. Method

3.1. Architecture Overview

Hyper-ARM consists of four main components: a Frozen Visual Encoder (E), a Temporal Context Module (optional for RL), a Hypernetwork-Generated Query Encoder (QE_{psi}), and a Sparse Memory Bank (H).

Frozen Visual Encoder (The Key Source): A CNN-VAE encodes raw pixels x_t into a latent vector z_t in R^d . This encoder is pre-trained and then frozen. This ensures that the "Key" space remains stable over time, preventing the invalidation of stored memories—a common failure mode in Neural Episodic Control (NEC).

$$z_t = E(x_t), \text{grad}(E) = 0$$

Temporal Context Module (RL Specific): To address the critique that single-frame queries lack temporal information, Hyper-ARM incorporates temporal context strictly for the Query generation, not the Key generation.

Frame Stacking: Following DQN conventions, the input x_t consists of 4–8 stacked frames.

Recurrent Branch: Optionally, velocity data or hidden states can be processed via a lightweight LSTM/GRU branch. The output of this branch is concatenated with the CNN features before entering the Query modulation MLP. This allows the Query to reflect "where we are going" while the Key reflects "where we are."

Hypernetwork-Generated Query Encoder: Unlike standard attention where Queries are fixed projections, our Query Encoder is dynamic.

Centroid Routing: The memory bank latents are clustered via online k-means to produce centroids $C = \{c_1, \dots, c_K\}$.

Hypernetwork: A hypernetwork h_{ψ} takes the top-k nearest centroids to the current state as input. It generates the weights θ_t for the Query Target Network.

Modulation: The Target Network (with weights θ_t) processes the visual/temporal features to produce a residual modulation vector Δ_{q_t} .

$$\theta_t = h_{\psi}(\text{top-k}(C, z_t))$$

$$\Delta_{q_t} = \text{TargetNet}_{\theta_t}(\text{CNN}(x_t) \text{ concat } \text{RNN}(h_{t-1}))$$

This vector is added residually to the frozen latent z_t to form the effective query vector q_t , which is then L2 normalized:

$$q_t = \text{Norm}(z_t + \Delta_{q_t})$$

This design allows the agent to learn how to attend to specific features without altering the underlying perceptual representation used for memory indexing.

Memory Bank Dynamics & Insertion: The memory H stores tuples (k_i, v_i) . To maintain sparsity and relevance:

Insertion: New memories (z_t, v_t) are inserted only if the cosine similarity between z_t and its 1-Nearest Neighbor (1-NN) in H is below a threshold τ_{sim} .

Stability: Since E is frozen, z_t serves as a stable address. Insertion happens post-encoder to ensure key consistency.

3.2. EQV Attention Mechanism

Standard attention utilizes Query, Key, and Value (QKV) projections derived from the same latent source. In Hyper-ARM, the mechanism is distinct (EQV: Encoder-Query-Value):

Encoder (E): Provides the static Key $k \approx z_t$. Since E is frozen, keys for identical observations remain identical across training.

Query (Q): Provides the dynamic projection q_t . The learned residual Δ_{q_t} shifts the query in

latent space to retrieve specific memories without moving the key itself.

Value (V): Retrieved action estimates v_i .

Decoupling Visual and Functional Similarity:

Disambiguation: Visually identical states (same k) can produce distinct queries ($q_a \neq q_b$) via contextual modulation Δ_{q_t} (driven by temporal context or hypernetwork routing), retrieving different memories.

Generalization: Visually distinct states can produce similar queries, retrieving the same memory despite pixel-level differences.

Attention weights α_i are computed via cosine similarity between the modulated query q_t and stored keys k_i . The retrieved value m_t is combined with the standard policy output via a residual connection.

$$\alpha_i = \exp(q_t \cdot k_i / \tau) / \sum_j \exp(q_t \cdot k_j / \tau)$$

The retrieved value m_t is the weighted sum of stored values:

$$m_t = \sum_i \alpha_i v_i$$

This retrieval is combined with the standard policy output via a residual connection or fusion layer:

$$\pi(a_t | z_t) = \text{softmax}(W_\pi z_t + \lambda m_t)$$

3.3. Multi-Hop Retrieval via Stacking

Hyper-ARM supports stacking EQV blocks to increase reasoning depth.

Residual Query Propagation: In a stacked configuration (Layer $L+1$), the input latent is the residual output from Layer L ($z_t(L+1) = z_t(L) + \Delta_{q_t}(L)$).

Static Keys: The initial Key source (E) remains static across all layers.

Accumulated Shift: The residual additions accumulate, effectively shifting the query into new regions of the latent manifold for the next layer's lookup. This allows the system to retrieve memories based on prior retrievals (e.g., "Retrieve state where I had the key, then retrieve state where I used the key") without requiring explicit recurrent hidden states or transformer self-attention re-projections.

Normalization is applied per layer to prevent magnitude explosion.

3.4. Training Objective

We optimize using a PPO loss L_{PPO} combined with a memory consistency loss L_{mem} . Gradients flow through the Hypernetwork and Query Target Network, but not through E .

$$L = L_{\text{PPO}} + \beta * L_{\text{mem}}$$

In Supervised Learning (SL) settings, the Query Encoder is trained using a frozen Value Encoder as a guidance signal (distillation), ensuring the Query Encoder learns to retrieve memories that align with robust parametric predictions.

4. Experiments

4.1. Empirical Validation: Vision Classification (SL)

To validate the core ARM mechanism independent of RL complexity, we implemented Hyper-ARM on MNIST and Fashion-MNIST benchmarks using a frozen Autoencoder and an external vector database.

- **Setup:** A Parallel Query Encoder (Residual CNN) was trained to modulate frozen latents for kNN retrieval. The CNN used minimal capacity (8 filters per layer).
- **MNIST Results:** The system achieved 98.2% accuracy, outperforming the frozen baseline encoder by 1.5%.
- **Fashion-MNIST Results:** The system achieved 92.0% accuracy, outperforming the frozen baseline (86.0%) by 6.0%.
- **Significance:** The larger gain on Fashion-MNIST suggests that Hyper-ARM provides greater benefits on visually complex tasks where parametric encoders struggle to generalize. The success with low filter counts confirms that the EQV mechanism offloads representational burden to the memory bank.

4.2. Proposed RL Evaluation: Atari 2600

Note: The following metrics represent preliminary runs and proposed benchmark targets to be fully validated in subsequent iterations.

We propose evaluating Hyper-ARM on the Atari 2600 benchmark (57 games) using the Gymnasium interface, comparing against DQN, R2D2, and Decision Transformer (DT).

- **Sample Efficiency Hypothesis:** Based on preliminary runs in *Venture* and *Montezuma's Revenge*, Hyper-ARM is projected to reach 80% of human normalized score within 10M steps, whereas R2D2 typically requires 40M steps.
- **Memory-Intensive Subset:** We anticipate significant gains in games requiring disambiguation of identical states (e.g., *Pitfall*, *Frostbite*).
- **Table 1 (Preliminary/Target Metrics):**
 - *Model:* Hyper-ARM (Ours) | *Mean:* ~115.8% | *Memory-Intensive:* ~88.7%
(These figures represent early convergence trends and are subject to full 50M step validation.)

4.3. Proposed Key Ablations

To rigorously isolate the contributions of Hyper-ARM, we propose the following ablation studies for the full RL benchmark:

1. **Frozen vs. Trainable Encoder:** Comparing the standard frozen EE against a trainable encoder to quantify memory address stability vs. representation adaptability. We hypothesize trainable encoders will cause catastrophic forgetting in the memory bank.
2. **Temporal Context Branch:** Removing the RNN/Frame-Stack input to the Query Encoder to measure the reliance on temporal history for disambiguation.
3. **Memory Insertion Threshold (τ_{sim}):** Varying the cosine similarity threshold for memory insertion to determine the optimal sparsity-density trade-off.
4. **Hypernetwork Routing:** Removing the hypernetwork (using a single shared policy head) to verify the necessity of latent specialization (MoE) for disjoint state manifolds.

5. **Multi-Hop Depth:** Stacking 1 vs. 3 vs. 5 EQV blocks to measure diminishing returns on reasoning depth versus computational cost.

4.4. Qualitative Analysis: Disambiguation Visualization

To verify the Visual vs. Functional Similarity hypothesis, we analyzed latent spaces in *Venture*.

- **t-SNE Projection:** Pairs of visually identical states (same room, different key status) overlap completely in Frozen Latents (z). However, in Modulated Queries (q), the pairs are distinctly separated. This confirms that the Query Encoder successfully modulates the latent based on context (temporal/memory) to retrieve different memories.

5. Discussion

Mechanistic Interpretability: One advantage of Hyper-ARM is interpretability. By inspecting the memory bank, we can see exactly which past experiences influenced a decision. The EQV distinction clarifies why this stability is achieved: by freezing the Encoder, we decouple *what we see* from *what we remember about it*.

Human-Like Associative Reasoning: The architecture mirrors human cognitive processes where contextual cues (Query) override raw sensory input (Encoder) to retrieve relevant memories. This suggests that decoupling perception from attention may be a general principle for efficient intelligence.

Parameter Efficiency: Our vision experiments demonstrate that significant accuracy gains can be achieved with minimal parameter overhead by leveraging non-parametric memory. This contrasts with standard scaling laws that require increasing model depth for performance gains.

Implications for PCG Pipelines: The sample efficiency and generalization capabilities of Hyper-ARM make it suitable for automated testing pipelines where thousands of unique environments must be validated daily. By decoupling perception from memory, agents can adapt to novel level configurations using prior experience rather than random exploration. standard scaling laws that require increasing model depth for performance gains.

Scaling Laws: Preliminary experiments suggest that stacking EQV blocks yields logarithmic improvements in performance on tasks requiring multi-step reasoning (e.g., *Montezuma's Revenge*), whereas scaling parameter count yields diminishing returns.

Future Work: World-ARM: While current Values are weighted sums, future iterations will explore learned Value residuals to enable full latent dynamics prediction, effectively creating a non-hallucinatory world model grounded in associative memory. This aligns with recent work on stable representation learning to prevent collapse during long-horizon prediction.

6. Conclusion

We presented Hyper-ARM, a reinforcement learning architecture that integrates associative memory with hypernetwork-based routing. Derived from practical constraints in developing an AI for *Color Switch*, Hyper-ARM offers a principled solution to perceptual aliasing and sample inefficiency. We

have validated the core EQV mechanism on supervised vision tasks, demonstrating significant accuracy gains with lightweight query networks. Future work will focus on the full-scale Atari validation proposed in Section 4.2 and the development of World-ARM, extending this memory mechanism to predict future latent states rather than just actions.

References

- Mnih, V., et al. (2015). Human-level control through deep reinforcement learning.
- Pritzel, A., et al. (2017). Neural Episodic Control.
- Ha, D. & Schmidhuber, J. (2018). World Models.
- Shazeer, N., et al. (2017). Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer.
- Vaswani, A., et al. (2017). Attention is All You Need.
- Xiao, H., et al. (2017). Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms.
- MERLIN (Rae et al., 2018): Memory-augmented RL with episodic control.
- Agent57 (Badia et al., 2020): Uses intrinsic motivation and memory to solve hard exploration games (Montezuma).
- Grill, J. B., et al. (2020). Bootstrap Your Own Latent-A New Approach to Self-Supervised Learning.